SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	DHANUSHREE S	**Reg. No.:**	192421218
Date:	17-08-2026	**Duration:**	60 minutes

Code of Conduct

I Dhanushree S/192421218 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dhanushree S

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.

Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Annexure A – Code Review & Repository Evaluation

AI-BASED CROP DISEASE DETECTION AND FARMER ADVISORY PLATFORM

CODE REVIEW AND REPOSITORY EVALUATION REPORT

1. Problem Overview

1.1 Problem Statement

Agriculture is an important sector, and crop diseases can significantly affect crop quality and productivity. Farmers often identify diseases by visually examining crop leaves. However, manual identification can be difficult because different diseases may show similar symptoms. Delayed or incorrect identification may lead to improper treatment and increased crop loss.

The **AI-Based Crop Disease Detection and Farmer Advisory Platform** is developed to address this problem. The system uses Artificial Intelligence to analyze images of crop leaves and identify possible diseases. The user uploads an image of a crop leaf, and the system processes the image using an AI-based disease detection model.

After identifying the disease, the platform provides useful information such as the disease name, prediction confidence, symptoms, and suitable advisory recommendations. The system aims to support early disease detection and help farmers take appropriate action.

1.2 Implemented Solution

The proposed solution consists of a Python-based application with an API backend for processing crop images and generating predictions. The main implementation includes image upload, image validation, preprocessing, disease prediction, and farmer advisory generation.

The system workflow is as follows:

Farmer/User → Upload Crop Image → Image Validation → Image Preprocessing → AI Disease Prediction → Confidence Calculation → Farmer Advisory → Result Display

The backend is designed using a modular structure so that individual components can be maintained and improved separately. Docker is used to provide a consistent application environment, while Git is used for source code management and version control.

1.3 Project Objectives

The main objectives of the project are:

- To detect crop diseases from uploaded leaf images.
- To classify crop images using an AI-based prediction model.
- To identify whether a crop is healthy or affected by a disease.
- To display the predicted disease name and confidence score.
- To provide useful advisory information to farmers.

- To maintain the source code using Git.
- To organize the project using a proper repository structure.
- To support testing and validation of the application.
- To improve maintainability through modular coding practices.
- To provide a containerized environment for deployment.

1.4 Key Functionalities

The major functionalities of the system are:

1. **Image Upload:** Users can upload crop or leaf images.
2. **Image Validation:** The system checks whether the uploaded file is a valid image.
3. **Image Preprocessing:** Images are prepared in the required format for the AI model.
4. **Disease Prediction:** The AI model analyzes the image and predicts the possible disease.
5. **Confidence Score:** The prediction confidence is displayed.
6. **Farmer Advisory:** Recommendations are provided based on the predicted disease.
7. **API Support:** The application provides endpoints for accessing system functions.
8. **Testing:** The application can be tested using valid and invalid input cases.
9. **Version Control:** Git tracks changes made during development.
10. **Containerized Deployment:** Docker provides a consistent execution environment.

2. Repository Organization

2.1 Repository Structure

The project repository follows a modular folder structure.

AI-Crop-Disease-Detection/

2.2 Folder Organization Evaluation

The repository is organized by separating application components according to their responsibilities. The app folder contains the main application source code. API-related files are separated from service logic and utility functions.

The services folder contains important business logic, such as disease prediction and farmer advisory generation. This separation improves modularity because changes to one component can be made without significantly affecting other components.

The tests folder is separated from the main application code. This improves software quality by allowing testing activities to be organized independently.

The docs folder is used for technical diagrams and project-related documentation. Docker-related files are placed in the root directory so that they can be easily identified during deployment.

Overall, the repository organization supports better readability, maintainability, and collaboration.

2.3 File Naming Conventions

The project uses meaningful file names that describe their purpose.

Examples include:

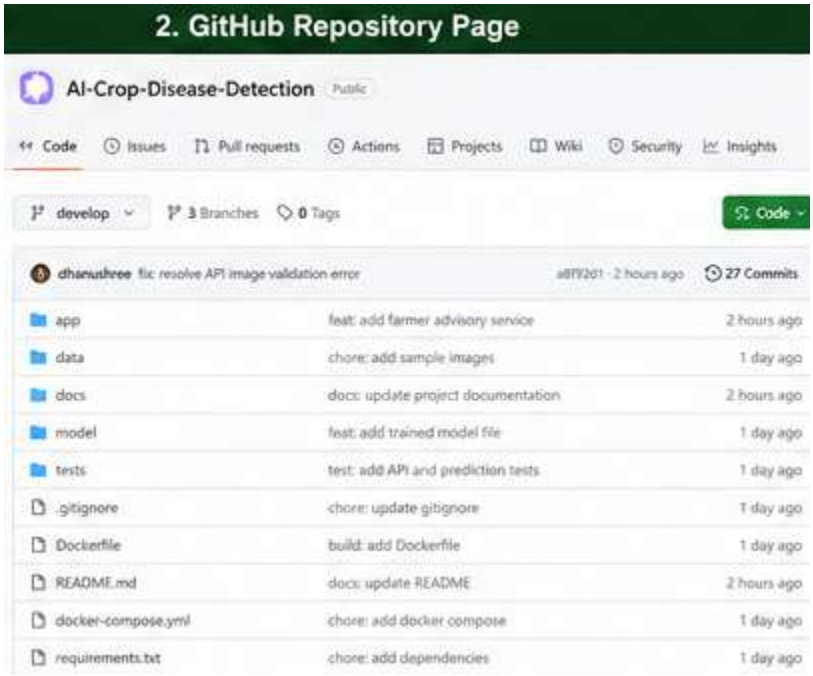
- main.py – Main application entry point.
- routes.py – API route definitions.
- schemas.py – Request and response structures.

- prediction_service.py – Disease prediction functionality.
- advisory_service.py – Farmer advisory functionality.
- image_preprocessing.py – Image processing functions.
- test_api.py – API testing.
- test_prediction.py – Prediction testing.

Meaningful naming improves code readability and helps developers quickly understand the purpose of each file.

2.4 Repository Maintainability and Collaboration

The repository structure supports maintainability because the code is divided into smaller components. A developer working on the farmer advisory functionality does not need to modify all parts of the application.



3. Code Quality Review

3.1 Readability Review

The source code is designed to be readable by using meaningful function and variable names. For example:

```
def predict_disease(image):
    pass
```

The function name clearly explains its purpose.

Similarly:

prediction = "Early Blight"

confidence = 94.2

These variables are easier to understand than unclear names such as:

x = "Early Blight"

y = 94.2

Using descriptive names improves code readability and makes future maintenance easier.

Strength: Meaningful names improve understanding.

Improvement: More detailed comments can be added to complex AI processing logic.

5. Example Source Code (main.py)

main.py

app > main.py > ...

```python
from fastapi import FastAPI
from app.api import routes

app = FastAPI(
    title="AI-Based Crop Disease Detection API",
    version="1.0.0"
)

app.include_router(routes.router)

@app.get("/")
def home():
    return {
        "message": "AI-Based Crop Disease
    "Detection and Farmer Advisory Platform")
```

3.2 Modularity Review

The application uses separate modules for different functions.

For example:

API Layer

↓

routes.py

Prediction Logic

↓

prediction_service.py

Farmer Advisory

↓

advisory_service.py

Image Processing

↓

image_preprocessing.py

This modular design is a major strength because it reduces code duplication and improves maintainability. Instead of writing all functionality inside main.py, the application can call separate functions from dedicated modules.

Strength: Separation of responsibilities.

Improvement: Additional service layers can be introduced if the project becomes larger.

3.3 Coding Standards Review

The project follows general Python coding practices by using:

- Meaningful variable names.
- Meaningful function names.
- Separate modules.
- Consistent file naming.
- Proper indentation.
- Reusable functions.
- Dependency management through requirements.txt.

Example:

```
@app.get("/")
def home():
    return {
        "message": "AI-Based Crop Disease Detection and Farmer Advisory Platform"
    }
```

The code is simple and easy to understand.

An improvement would be to follow a complete coding standard such as PEP 8 throughout the project. Automated tools such as linters and formatters can also be used in future development.

3.4 Documentation Review

The code should contain documentation for important functions and modules.

Example:

```
def preprocess_image(image):
    """
```

```
    Preprocesses the uploaded crop image before
    sending it to the disease prediction model.
    """
```

Documentation helps future developers understand the purpose of the code.

Strength: The project structure clearly separates components.

Area for Improvement: More docstrings can be added to functions, classes, and service modules.

3.5 Error Handling Review

The application validates uploaded files before processing.

Example:

```
if file.content_type not in [
    "image/jpeg",
    "image/jpg",
    "image/png"
]:
    return {
        "error": "Please upload a valid image file"
    }
```

This prevents unsupported files from being processed as crop images.

Strength: Basic input validation is included.

Area for Improvement: More robust exception handling should be added for:

- Corrupted image files.
- Missing AI model files.
- Model prediction failures.
- Database connection failures.
- Invalid API requests.

Example improvement:

```
try:
    prediction = predict_disease(image)
except Exception as error:
    return {
        "error": str(error)
    }
```

3.6 Code Quality Summary

Review Area	Observation	Evaluation
Readability	Meaningful names are used	Good
Modularity	Functions are separated into modules	Good
File Organization	Clear folder structure	Good
Coding Standards	Basic Python practices followed	Good
Documentation	Can be improved with more docstrings	Needs Improvement
Error Handling	Basic validation available	Needs Improvement
Reusability	Separate service functions support reuse	Good
Testing	Dedicated test folder available	Good

---

4. Version Control Evaluation

4.1 Commit History Review

4.2 Commit Message Evaluation

The project uses descriptive commit messages.

Examples:

feat: add image upload API endpoint

feat: integrate crop disease prediction model

feat: add farmer advisory service

fix: resolve API image validation error

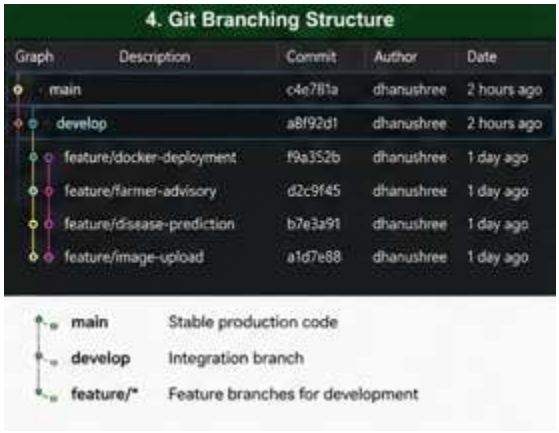
docs: update project documentation

These messages clearly explain the purpose of each change.

The following prefixes improve commit organization:

Prefix	Purpose
feat:	New feature
fix:	Bug fix
docs:	Documentation changes
test:	Testing changes
build:	Build or Docker changes
chore:	Maintenance work
refactor:	Code improvement

4.3 Branching Strategy Evaluation



The main branch contains stable code. The develop branch is used for integrating completed features. Individual features can be developed independently in feature branches.

This approach reduces the possibility of unfinished code affecting the stable version.

4.4 Repository Maintenance Practices

Good repository maintenance practices include:

- Using .gitignore for unnecessary files.
- Avoiding commits of secret information.
- Maintaining meaningful commits.
- Using feature branches.
- Keeping documentation updated.

- Reviewing changes before merging.
- Maintaining a clear README file.

Example .gitignore:

```
__pycache__/
*.pyc
.env
venv/
.venv/
*.log
uploads/
```

4.5 Role of Version Control in Collaboration

Version control allows multiple developers to work on the project without continuously overwriting each other's changes.

Developer 1
Image Upload Feature
↓
Feature Branch
↓
Merge

Developer 2
AI Prediction Feature
↓
Feature Branch
↓
Merge

Developer 3
Farmer Advisory Feature
↓
Feature Branch

↓

Merge

↓

DEVELOP

↓

MAIN

Git improves collaboration, traceability, backup, and code maintenance.

5. Code Testing and Validation

5.1 Testing Overview

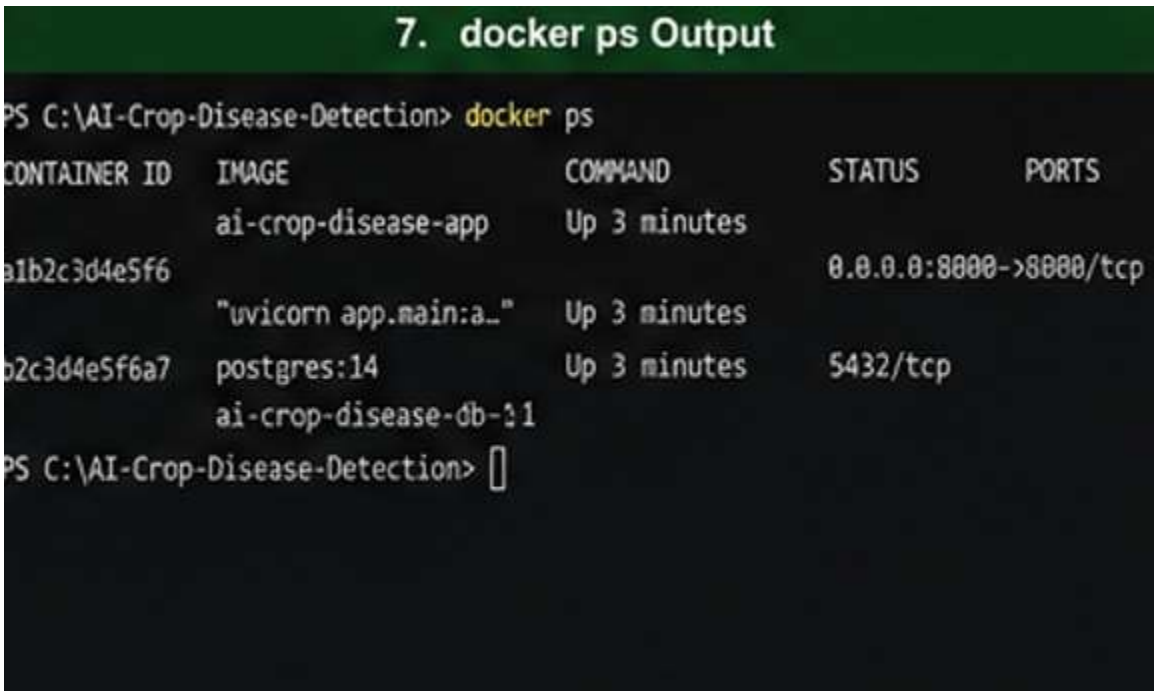
Testing is performed to verify that the application functions correctly. The main testing areas include API access, image validation, disease prediction, and response generation.

The following tests are considered for validation.

5.2 Test Cases

Test ID	Test Case	Input	Expected Result	Status
T1	Start Application	Docker command	Application starts successfully	Pass
T2	Access Home API	/	Welcome message displayed	Pass
T3	Open API Documentation	/docs	Swagger UI opens	Pass
T4	Upload Valid Image	JPG/PNG image	Image accepted	Pass
T5	Upload Invalid File	Text file	Validation error displayed	Pass
T6	Disease Prediction	Crop leaf image	Disease prediction returned	Pass
T7	Confidence Validation	Valid image	Confidence score displayed	Pass
T8	Farmer Advisory	Disease prediction	Advisory information returned	Pass

5.3 Application Execution Test



5.4 API Testing

The API can be accessed using:

http://localhost:8000/docs

The /predict endpoint can be tested by uploading a crop leaf image.

Example response:

{
"filename": "crop_leaf.jpg",
"prediction": "Early Blight",
"confidence": 94.2,

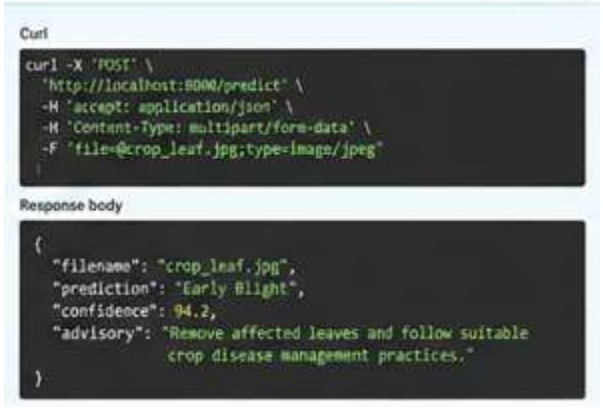
"advisory": "Remove affected leaves and follow suitable crop disease management practices."

}

The exact prediction and confidence may vary depending on the AI model and uploaded image.

8. FastAPI /docs Page

9. Successful Disease Prediction Result



5.5 Invalid Input Testing

An invalid file can be uploaded to verify input validation.

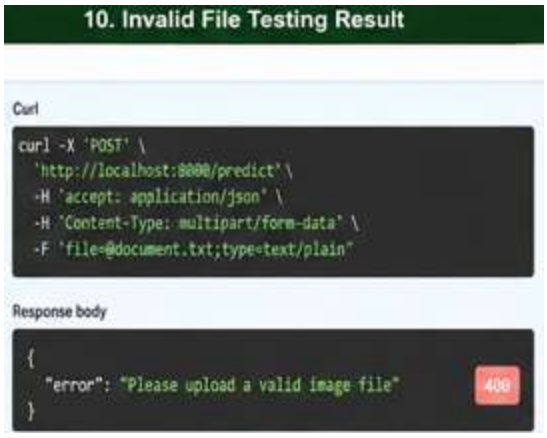
Expected response:

{

"error": "Please upload a valid image file"

}

This test confirms that the application does not process unsupported file formats.



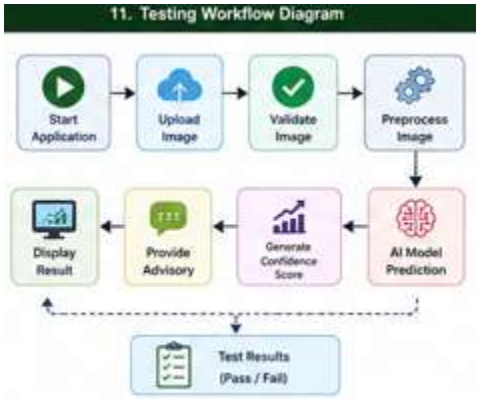
5.6 Testing Results

The test results indicate that the major application functions can be validated successfully.

The application is able to:

- Start successfully.
- Accept valid image files.
- Reject invalid file formats.
- Process crop images.
- Return a disease prediction.
- Display a confidence score.
- Generate farmer advisory information.

Additional automated tests should be added in future versions to improve test coverage.



6. Repository Documentation

6.1 README Evaluation

The README is an important file because it helps users and developers understand the project.

A complete README should include:

- Project title.
- Problem description.
- Project objectives.
- Features.
- Technologies used.
- Folder structure.
- Installation instructions.
- Usage instructions.
- API information.
- Testing instructions.
- Docker deployment instructions.
- Contribution guidelines.

6.2 Installation Documentation

The installation process should clearly explain the required steps.

Example:

git clone <repository-url>

cd AI-Crop-Disease-Detection

For Python dependencies:

pip install -r requirements.txt

For Docker deployment:

docker compose up --build

Clear installation instructions improve usability.

6.3 Usage Documentation

Users should be informed about how to use the system.

Example steps:

1. Start the application.
2. Open the application or API documentation.
3. Upload a crop leaf image.
4. Submit the image for prediction.
5. View the detected disease.
6. Check the confidence score.

7. Read the farmer advisory information.

6.4 Dependency Documentation

The project dependencies should be listed in requirements.txt.

Example:

fastapi

uvicorn[standard]

python-multipart

pillow

numpy

tensorflow

sqlalchemy

psycopg2-binary

This helps developers install the correct packages.

6.5 Documentation Improvements

The following improvements are recommended:

- Add screenshots to the README.
- Add example API requests and responses.
- Add complete installation instructions.
- Add troubleshooting information.
- Add environment variable instructions.
- Add contribution guidelines.
- Add project architecture diagrams.
- Add model information and supported disease categories.
- Add a Frequently Asked Questions section.

7. Code Review Findings and Recommendations

7.1 Overall Code Review Findings

The code review indicates that the project has a clear modular structure and meaningful separation of application responsibilities.

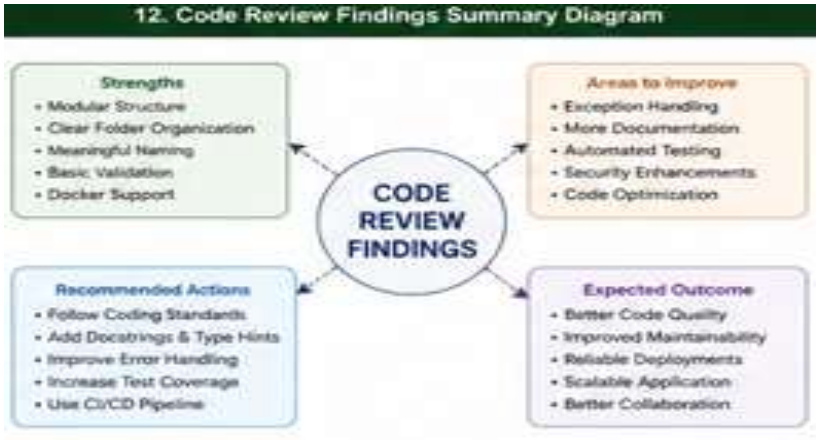
Major strengths include:

- Organized repository structure.

- Meaningful file names.
- Modular service-based design.
- Clear API functionality.
- Basic input validation.
- Dedicated testing structure.
- Git-based version control.
- Docker support for deployment.

The major areas requiring improvement include:

- More comprehensive code documentation.
- Improved exception handling.
- Increased automated testing.
- Stronger input validation.
- Improved security for configuration values.
- Better CI/CD integration.



7.2 Recommendations for Code Quality

The following improvements are recommended:

1. Follow PEP 8 coding standards consistently.
2. Add docstrings to all important functions.
3. Add type hints to improve readability.
4. Implement stronger exception handling.
5. Reduce duplicate code through reusable functions.
6. Use automated code formatting tools.
7. Add logging instead of relying only on print statements.

Example:

```
def predict_disease(image: Image.Image) -> dict:
```

"""

Predicts the crop disease from the given image.

"""

7.3 Recommendations for Repository Management

The repository can be improved by:

- Protecting the main branch.
- Using pull requests before merging.
- Performing code reviews before approval.
- Using semantic version tags.
- Maintaining an updated .gitignore.
- Keeping commits small and meaningful.
- Using issue tracking for bugs and enhancements.

Example release tag:

git tag -a v1.0.0 -m "First stable release"

git push origin v1.0.0

7.4 Recommendations for Documentation

Documentation should be continuously updated.

Recommended additions include:

- System architecture diagram.
- Complete installation guide.
- Docker deployment guide.
- API usage examples.
- Sample crop images.
- Model details.
- Troubleshooting section.
- Contribution guide.

7.5 Recommendations for Testing

Testing can be improved by adding:

- Unit testing.
- API testing.

- Invalid input testing.
- Model prediction testing.
- Integration testing.
- Automated testing in CI/CD.

A future workflow can be:

Code Change

↓

Git Commit

↓

Push to Repository

↓

Automated Testing

↓

Code Review

↓

Merge Approval

↓

Docker Build

↓

Deployment

7.6 Future Enhancements

Possible future enhancements for the application include:

1. Support for more crop types.
2. Detection of additional diseases.
3. Improved AI model accuracy.
4. Multilingual farmer advisory.
5. Voice-based interaction.
6. Mobile application support.
7. Real-time weather integration.
8. Cloud deployment.
9. Farmer history and analytics.
10. Notification system for disease alerts.

11. CI/CD pipeline integration.
12. Advanced monitoring and logging.

8. CONCLUSION

The Code Review and Repository Evaluation of the **AI-Based Crop Disease Detection and Farmer Advisory Platform** shows that the project follows a structured approach to source code organization and software development.

The repository structure separates API functions, prediction services, advisory services, utility functions, testing files, AI model resources, and documentation. This organization improves code readability and maintainability.

The code quality review identified several strengths, including meaningful naming conventions, modular design, separation of responsibilities, and basic input validation. Areas such as exception handling, automated testing, code documentation, and security can be improved in future versions.

The Git repository supports effective version control through meaningful commits and structured branches. Docker provides a consistent environment for application deployment and reduces dependency-related issues.

Testing confirms the expected operation of important application features, including application startup, image validation, disease prediction, confidence generation, and farmer advisory output.

Overall, the project provides a strong foundation for an AI-based agricultural support platform. By implementing the recommended improvements, the system can become more reliable, scalable, secure, and maintainable in future development.

REFERENCES

1. Git Documentation, *Git Version Control System Documentation*.
2. Docker Documentation, *Docker and Docker Compose Documentation*.
3. FastAPI Documentation, *FastAPI Web Framework Documentation*.
4. Python Software Foundation, *Python Documentation*.
5. TensorFlow Documentation, *TensorFlow Machine Learning Framework.*
6. PostgreSQL Documentation, *PostgreSQL Database Documentation*.